

Plant Watering System

Team Members: Pantea Mehdigholi Khani,
Zeinab Mollaei, Venus Jamali

SDD Author: Venus Jamali

This document presents the SDD for the Arduino-based Plant Watering System. It describes the software architecture, module organization, hardware interfaces, data flow, and design decisions that define the system's implementation. The document serves as a technical reference for understanding, developing, maintaining, and extending the software throughout the project's lifecycle.

Contents

1. Project Overview	5
1.1 Introduction.....	5
1.2 General System Description	5
1.3 Main Features	5
1.4 Operating Principle	6
1.5 User Interaction	6
1.6 Operating Modes	6
1.6.1 Moisture Mode	6
1.6.2 Timer Mode	7
1.6.3 View Status	7
1.7 Persistent Configuration	7
1.8 User Configuration.....	8
1.9 Water Pump Control.....	8
1.10 Soil Moisture Monitoring	8
1.11 Software Design Philosophy	8
1.12 Expandability	9
2. Complete Functional Specification.....	10
2.1 Startup Behavior	10
2.2 Home Screen	10
2.3 Main Menu.....	10
2.4 Moisture Mode	10
2.4.1 LCD Display	10
2.4.2 Moisture Mode Runtime Behavior	11
2.5 Timer Mode	11
2.5.1 Timer Mode Runtime Behavior	12
2.6 Status Screen	12
2.7 Tank Screen.....	13
2.8 Watering Screen.....	14
2.9 Keypad Behavior	14
2.10 LCD Behavior	14
2.11 EEPROM Behavior	14
2.12 Error Handling	14
2.13 Software Timing Requirements	14
2.14 Operating Modes Summary	15
2.15 Future Expandability	15

3. Hardware Description.....	16
3.1 Overall Hardware Architecture.....	16
3.1.1 Input Subsystem.....	16
3.1.2 Processing Subsystem	16
3.1.3 Output Subsystem.....	16
3.1.4 Power Subsystem.....	16
3.2 Protection Features.....	16
4. Pin Mapping	18
5. Libraries.....	19
5.1 Development Environment.....	19
5.2 PlatformIO Libraries.....	19
5.2.1 LiquidCrystal_I2C.....	19
5.2.2 Keypad Library.....	19
5.2.3 EEPROM Library	19
5.2.4 Wire Library.....	20
5.2.5 Arduino Core	20
5.3 PlatformIO Configuration.....	20
6. PlatformIO Project Structure	21
6.1 include/.....	21
6.2 lib/	21
6.3 src/	22
6.4 test/	22
6.5 docs/	22
7. Folder Structure	23
7.1 Structure overview	23
7.2 Folder Responsibilities.....	24
7.2.1 LCDManager	24
7.2.2 KeypadManager	24
7.2.3 SensorManager	24
7.2.4 PumpController	25
7.2.5 EEPROMManager	25
7.2.6 TimerManager.....	25
7.2.7 MenuManager	25
7.2.8 ModeManager	25
7.2.9 StatusManager.....	25
7.2.10 Hardware Layer.....	26

8. Software Architecture	27
8.1 Overview.....	27
8.2 Layer Responsibilities.....	28
8.2.1 Application Layer.....	28
8.2.2 Logic Layer.....	28
8.2.3 Service Layer.....	28
8.2.4 Hardware Abstraction Layer (HAL).....	28
8.2.5 Physical Hardware Layer	28
9. Finite-State Machine (FSM).....	30
9.1 Purpose.....	30
9.2 Complete State List	30
9.3 FSM Diagram (Logical)	31
9.4 State Descriptions	32
9.4.1 STATE_BOOT	32
9.4.2 STATE_HOME.....	32
9.4.3 STATE_MENU.....	32
9.4.4 STATE_MOISTURE_THRESHOLD	32
9.4.5 STATE_MOISTURE_DURATION	33
9.4.6 STATE_TIMER_INTERVAL.....	33
9.4.7 STATE_TIMER_DURATION	33
9.4.8 STATE_SAVE_SETTINGS.....	33
9.4.9 STATE_STATUS	33
9.4.10 STATE_WATERING	33
9.4.11 STATE_TANK_MENU_TRANSITION	33
9.4.12 STATE_TANK_MENU	33
9.4.13 STATE_REFILL_CONFIRM.....	34
9.4.14 STATE_SET_TANK_CAPACITY	34
10. EEPROM Memory Layout.....	35
10.1 EEPROM Signature.....	35
10.2 Factory Defaults	35
11. Moisture Sensor Processing.....	36
11.1 Reading Method.....	36
11.2 Filtering.....	36
11.3 Calibration	36
11.4 Moisture Mode Decision	36
11.5 Sensor Fault Detection	36

12. Pump Control Logic.....	37
12.1 General description	37
12.2 Pump Activation	37
12.3 Watering Cycle	37
12.4 Tank Volume Estimation	37
12.5 Cooldown Protection	38
12.6 Mutual Exclusion.....	38
12.7 Safety on Startup	38
13. Algorithms	39
13.1 Timer Mode Algorithm	39
13.2 Moisture Mode Algorithm	39
13.3 Tank Volume Algorithm.....	40
14. Startup Sequence	41
15. Shutdown Behavior.....	42
16. Error Handling.....	43
16.1 Invalid User Input.....	43
16.2 EEPROM Initialization Failure	43
16.3 Sensor Fault (Future Support)	43
16.4 Low Water Condition	43
16.5 Pump Protection	43
17. Input Validation	44
18. Memory Constraints	45

1. Project Overview

1.1 Introduction

This project implements a complete Arduino Nano-based plant watering system capable of automatically irrigating a plant using either soil moisture measurements or a user-configurable watering schedule. The software has been designed as a structured embedded application featuring a modular architecture, persistent configuration storage, and an intuitive user interface. This document describes the software design, architectural decisions, and implementation details that enable the system's functionality, maintainability, and future extensibility.

The entire software is developed using PlatformIO in Visual Studio Code, following modern C++ programming practices while remaining compatible with the Arduino framework.

The user never needs to modify the source code. Every configurable parameter is adjustable directly from the keypad, and all settings are stored permanently inside EEPROM so they remain available after power loss.

The project demonstrates the integration of multiple embedded subsystems including:

- Human-machine interface
- Sensor acquisition
- Digital signal filtering
- State machine implementation
- EEPROM data storage
- Actuator control
- Timing algorithms
- Modular software engineering
- Error handling
- Embedded user experience

1.2 General System Description

The plant watering system continuously monitors the operating mode selected by the user. The software supports two independent automatic watering modes: "Moisture Mode" and "Timer Mode". Only one operating mode is active at any given time. The active mode is selected by the user through the LCD menu and remains active until another mode is selected. The system continuously executes background monitoring while waiting for user interaction.

1.3 Main Features

The system provides the following major features:

- Automatic watering based on soil moisture.
- Automatic watering based on elapsed time.
- Fully menu-driven user interface.
- LCD-guided configuration process.
- Numeric input through a 4×4 keypad.
- Permanent EEPROM storage.
- Automatic restoration of settings after reboot.
- Continuous system monitoring.
- Pump protection.

- Input validation.
- Sensor filtering.
- Software debouncing.
- Structured finite-state machine.
- Layered software architecture.
- Modular source code.
- Expandable project structure.
- Automatic estimation of remaining tank water volume.
- User-configurable initial tank capacity.
- Low-water warning indication.
- Automatic prevention of watering when the tank is empty.

1.4 Operating Principle

The Arduino continuously performs four major tasks:

1. Read user input.
2. Update the LCD interface.
3. Monitor sensors.
4. Execute watering logic.

These tasks execute repeatedly inside the main loop without blocking the processor. The software avoids long delays wherever possible so that user input remains responsive while background monitoring continues simultaneously.

1.5 User Interaction

The entire system is operated through:

- one 16×2 LCD display
- one 4×4 keypad

The LCD serves as the only graphical interface for the user.

The keypad is used for:

- opening the menu
- selecting options
- entering numerical values
- confirming settings
- cancelling operations
- navigating between screens

1.6 Operating Modes

The software supports three primary menu options.

1.6.1 Moisture Mode

In Moisture Mode, the Arduino continuously measures soil moisture using the moisture sensor. The measured value is compared against a user-defined moisture threshold. The user defines:

- moisture threshold watering level
- pump operating duration

Whenever the moisture level falls below the configured threshold, the Arduino activates the water pump for a user-defined watering duration. After watering, the system waits for a stabilization period before reading the sensor again to allow water to penetrate the soil and prevent repeated immediate watering. This mode automatically adapts watering frequency according to the actual condition of the soil.

1.6.2 Timer Mode

In Timer Mode, watering occurs strictly according to time rather than soil moisture.

The user specifies:

- watering interval
- pump operating duration

The Arduino continuously measures elapsed time. Whenever the configured interval expires, the pump is activated for the configured duration. After watering, the timer restarts automatically. This mode is useful when watering must occur on a predictable schedule regardless of soil measurements.

1.6.3 View Status

The View Status screen provides real-time information about the current operating condition of the system.

The LCD displays important runtime information such as:

- currently selected mode
- configured settings
- latest moisture reading
- remaining time until next watering (Timer Mode)
- stored moisture threshold (Moisture Mode)
- watering duration
- Estimated remaining tank water

This screen allows the user to verify system configuration without modifying any settings.

1.7 Persistent Configuration

Every configurable parameter entered by the user is stored inside the Arduino's internal EEPROM.

Examples include:

- selected operating mode
- moisture threshold
- watering duration
- watering interval
- calibration values (if applicable)
- Current estimated tank volume
- Configured full tank capacity

After power loss, unplugging the Arduino, or restarting the device, all settings are automatically restored without requiring the user to reconfigure the system.

1.8 User Configuration

The following parameters are configurable entirely through the keypad:

Moisture Mode

- Moisture threshold
- Pump running duration

Timer Mode

- Watering interval
- Pump running duration

The software validates every entered value before accepting it. Invalid values are rejected with an appropriate LCD message.

1.9 Water Pump Control

The water pump is controlled through a relay module rather than being powered directly by the Arduino Nano. The Arduino output pin provides only the low-current control signal required to energize the relay, while the relay switches the external power supplied to the pump. This arrangement electrically isolates the microcontroller from the higher-current pump circuit, allowing the pump to be controlled safely without exceeding the current limitations of the Arduino's I/O pins.

The pump only operates when:

- watering conditions are satisfied
- the configured watering duration has not expired
- no software error prevents operation

The software guarantees that the pump cannot remain permanently ON due to programming errors.

1.10 Soil Moisture Monitoring

The soil moisture sensor continuously measures the moisture level of the soil.

To improve measurement reliability, the software processes the raw sensor readings before making watering decisions. This includes taking multiple samples, filtering noise, and converting the readings into a stable value suitable for comparison against the configured threshold.

Only the processed moisture value is used by the watering algorithm.

1.11 Software Design Philosophy

Rather than implementing all functionality inside a single Arduino sketch, the software follows a modular layered architecture.

Each subsystem is implemented independently, including:

- LCD management
- keypad handling
- menu system
- EEPROM management
- moisture sensor processing

- pump controller
- timer manager
- settings manager
- state machine
- validation logic
- utility functions

Each module exposes a clean interface while hiding its internal implementation from the rest of the system. This separation improves readability, simplifies debugging, reduces coupling between components, and allows individual modules to be extended or replaced without affecting the overall system.

1.12 Expandability

The software architecture supports future expansion with minimal modification to existing modules.

Potential future features include:

- OLED or graphical display support
- Multiple watering zones
- RTC (Real-Time Clock) scheduling
- Wi-Fi or Bluetooth connectivity
- Mobile application integration
- Automatic tank level sensor
- Temperature and humidity monitoring
- Multiple moisture sensors
- Multiple irrigation zones
- Additional menu options

2. Complete Functional Specification

2.1 Startup Behavior

When power is applied, the system performs an initialization sequence to prepare all hardware components and restore the previously configured operating parameters.

The startup sequence consists of the following steps:

1. Initialize the serial communication interface for debugging and monitoring purposes.
2. Initialize the LCD display interface.
3. Initialize the keypad interface for user input.
4. Initialize the soil moisture sensing subsystem.
5. Configure the relay control output used for pump activation.
6. Configure the system status indicators and warning LEDs.
7. Retrieve stored configuration data from EEPROM.
8. Validate the retrieved settings to ensure data integrity.
9. Load default configuration values if the stored EEPROM data is invalid.
10. Display the startup screen for approximately two seconds.
11. Transition the system to the Home Screen and begin normal operation.

2.2 Home Screen

The LCD normally remains on the Home Screen.

Home screen displays:

Row 1: Plant Watering

Row 2: A = Menu

The system continues monitoring sensors and timers while showing this screen. No watering functionality is paused while on the Home Screen.

2.3 Main Menu

When the user presses 'A' from the Home Screen, the system enters the Main Menu. LCD shows:

Row 1: 1:M 2:T

Row 2: 3:Status 4:Tank

the user presses 1 or 2 or 3 or 4 to enter the desired screen. Pressing D at any menu level returns one level back. Pressing D from the Home Screen has no effect.

2.4 Moisture Mode

2.4.1 LCD Display

Selecting option 1 enters Moisture Mode configuration.

LCD displays:

Row 1: Threshold (%)

User enters a number using the keypad.

Example:

35

The entered digits are shown live on the LCD on the second row. Pressing # means confirmation. Only ranges 1 to 100 are allowed. After confirmation, the configured moisture threshold is stored in EEPROM as an integer percentage value within the range of 0–100. The system then converts this value into the required internal representation for moisture comparison during operation. A confirmation message is displayed for approximately three seconds, after which the system returns to Moisture Mode and resumes normal operation.

Next screen:

Row 1: Pump Time (sec):

User enters watering duration. The number the user inputs is displayed on the screen on the second row.

Example:

5

seconds.

Ranges 1 to 60 are valid. Pressing # means validation. If # is pressed and the number which the user wants to confirm is not valid, the screen is cleared and the LCD displays the appropriate screen. Once the number is valid and verified, LCD displays:

Row 1: Settings Saved

For 4 seconds and then returns back to Home Screen. The selected operating mode now becomes "Moisture Mode" and remains active until changed.

2.4.2 Moisture Mode Runtime Behavior

The system continuously reads the soil moisture sensor. The moisture reading is converted to percentage. If the current moisture read is less than threshold, watering begins. Pump output becomes on and the green LED turns on. The water pump remains active exactly for the configured duration. Then both the water pump and the green LED turn off. The system then waits for a cooldown period before checking again. This prevents rapid repeated watering. Cooldown is configurable as a software constant.

2.5 Timer Mode

Selecting option 2 opens Timer Mode. LCD displays:

Row 1: Water Every

Row 2: Minutes

User enters the interval. Each number the user enters are shown on the screen on row 1.

Example:

180

Ranges 1-999 is valid. The fourth digit is ignored. Pressing # means confirmation. Once the number entered is valid, LCD goes to the next screen.

Row 1: "Pump Time (s)"

User enters watering duration. The watering duration entered by the user, each character/number entered is shown on screen on the second row.

Example:

8

Ranges 1 to 60 are allowed. Once a valid number is entered, LCD displays:

Row 1: Settings Saved

for 4 seconds and then returns back to Home Screen. Current operating mode becomes Timer Mode.

2.5.1 Timer Mode Runtime Behavior

The software records the current millis() value. When the configured interval has elapsed, water pump and the green LED turn on. After the configured durations has passed, both water pump and the green LED turn off again, and the timer resets. The timer does not drift due to LCD operations or keypad usage.

2.6 Status Screen

The View Status menu provides read-only information about the current system configuration. Selecting option 3 shows current information.

First page:

Row 1: Mode: current mode

Row 2: Tank: water left in the tank

Example:

Mode: Moisture

Tank: 950mL

or

Mode: Timer

Tank: 230 mL

Second Screen:

Row 1: Moisture: moisture sensor number in percentage

Row 2: Pump: pump status

Example:

Moisture: 42%

Pump: OFF

Caution: The moisture sensor number is read at any time no matter what the mode is. The difference between timer mode and moisture mode is in what way the plant is watered, but the moisture is read regardless of what the mode is.

Third Screen:

This page depends on what mode the system is on. If the system is on moisture mode, LCD displays:

Row 1: Threshold: threshold number in percentage (with the percentage icon)

Row 2: Time: the time duration the user has written for the water pump to be on (with sec written at the end of the number so we can see the unit of it)

Example:

Threshold: 35%

Time: 5 sec

If the system is on time mode, on the third screen LCD displays:

Row 1: Interval: The time interval the user has written which is spent between the time the plant is watered. The m unit is written after the number.

Row 2: Time: the time interval, the user input which is spent with the water pump on every time it waters the plant. The sec unit is written after the number.

Example:

Interval:180m

Time:8 sec

In the status screen, because of the size of LCD, pages are changed every three seconds (First page, second page, third page, then again from the start) up until the # key is pressed. After # LCD returns to Home Screen. No settings are modified from the Status screen.

2.7 Tank Screen

When the user presses 4 in the Menu Screen the tank screen will be visible. LCD displays:

Row 1: Refill Tank

Row 2: Set Tank Size

If 1 is pressed, LCD displays:

Row 1: Tank Filled?

In the middle of the page. If # is pressed for confirmation, *setRemainingWater = TankCapacity* and LCD shows "**Row 1:** Settings Saved" for three seconds and then returns back to Home Screen.

If 2 is pressed in the Tank Screen, LCD goes to another screen showing "Row 1: Tank Capacity" with row 2 nothing. User will enter a value between 1-9999. The number is stored in EEPROM. Factory default is 2000mL. This is the only place where 4-digit numbers are allowed. The fifth digit is ignored. After the valid number is entered, LCD goes back to Home Screen. Another way for the user to tell the system the water tank is filled, is by pressing key 'C'. If the system is on STATE_HOME or STAET_LOW_WATER and key 'C' is pressed, in that case the remainingWater is changed to tankCapacity.

2.8 Watering Screen

When the water pump is on, LCD displays:

Row 1: Watering

In the middle of the page. This is written as long as the water pump is on. Write after the pump turns off, this returns back to Home Screen.

2.9 Keypad Behavior

Numeric keys are used only for entering numeric values. For confirming current input # is pressed. For deleting previous digit * is pressed. For instance, if 123 is entered and then * is pressed, we'll have 12. Key "A" will open the main menu. Key "D" is for going back one level. If "D" is pressed when LCD is on Home Screen, nothing happens.

2.10 LCD Behavior

The display subsystem is designed to minimize unnecessary LCD updates. A dedicated Screen Manager tracks the currently displayed screen and refreshes only characters that have changed, avoiding full-screen redraws whenever possible. The `lcd.clear()` operation is used only when required for screen transitions and is never called repeatedly during normal operation, preventing visible display flicker. The cursor is enabled only during numeric input and remains hidden during all other system states. Text is centered on the display whenever practical, subject to the constraints of the 16×2 LCD.

2.11 EEPROM Behavior

The following parameters are saved:

1. Current Mode
2. Moisture Threshold
3. Pump Duration
4. Timer Interval
5. Calibration Values

Settings survives power loss. EEPROM writes only occur when settings actually change. Unnecessary EEPROM wear is avoided.

2.12 Error Handling

Invalid keypad input: Row 1→Invalid Input

Out-of-range values: Row 1→Invalid Value

Sensor disconnected: Row 1→Sensor Row 2→Disconnected

EEPROM corruption: Row 1→EEPROM Row 2→Corruption

Unexpected state: Row 1→Unexpected Row 2→State

Each error displays an informative LCD message which is written in from of each of them. The software recovers automatically whenever possible.

2.13 Software Timing Requirements

Temporary display screens (e.g., *Settings Saved*, *Invalid Value*, or *Try Again*) are implemented as dedicated finite-state machine (FSM) states rather than using blocking delays or loops.

Upon entering a temporary message state, the current system time is recorded using `millis()`, and the corresponding LCD message is rendered once. During each iteration of the main loop, the elapsed time is evaluated, and when the configured display duration has expired, the FSM automatically transitions to the designated destination state (e.g., the home screen or the previous input state). Throughout the display of temporary messages, background operations, including moisture sensing, keypad polling, and timer processing, continue to execute without interruption.

2.14 Operating Modes Summary

Exactly one operating mode is active at any moment.

Possible modes:

- Home
- Menu
- Moisture Configuration
- Timer Configuration
- Status
- Moisture Runtime
- Timer Runtime

The program transitions between these using a finite-state machine.

2.15 Future Expandability

Software architecture supports future addition of these below without requiring major restructuring.

- Water level sensor
- RTC module
- Temperature sensor
- Humidity sensor
- Bluetooth
- Wi-Fi
- Mobile application
- Additional menu items

3. Hardware Description

3.1 Overall Hardware Architecture

The system is centered around an Arduino Nano, which performs all sensing, decision making, user interaction, and actuator control. The hardware consists of four major subsystems:

3.1.1 Input Subsystem

- Soil moisture sensor
- 4×4 matrix keypad

These components provide environmental measurements and user commands.

3.1.2 Processing Subsystem

Arduino Nano

Responsibilities include:

- Reading the analog moisture sensor
- Scanning keypad rows and columns
- Updating LCD
- Executing finite-state machine
- Maintaining timers
- Controlling watering logic
- Saving settings into EEPROM

3.1.3 Output Subsystem

1. 16×2 I2C LCD. Displays:
 - Menus
 - Settings
 - Status
 - Error messages
 - Runtime information
2. Water Pump. Provides irrigation. Driven through a relay because the Arduino cannot directly supply the required current.
3. LED Indicators
 - Green LED: This one is on when pump is running.
 - Red LED: This one is on when the water left in the tank (which is calculated from the speed the water is poured into the plant and how long the water pump is on and how many times has it worked yet.) is less than 250mL. Also, this LED is on for system errors or sensor errors. When the error is for low water, the red LED is on constantly, but if it's on because of other errors, it flickers.

3.1.4 Power Subsystem

Arduino is powered through USB during development. Water pump is powered from an external supply appropriate for the selected pump (5V). The Arduino and pump supply share a common ground.

3.2 Protection Features

- Common ground between all supplies.

- relay
- Gate pull-down resistor.
- Flyback diode.
- External pump power supply.
- Current never passes through Arduino I/O pins directly.

4. Pin Mapping

Arduino Pin	Connected Device	Signal Type	Direction	Notes
A0	Soil Moisture Sensor	Analog Input	Input	Moisture reading (0–1023 ADC)
A4	16×2 LCD (I2C SDA)	I2C	Bidirectional	Shared I2C bus
A5	16×2 LCD (I2C SCL)	I2C	Output/Input	Shared I2C bus
D2	Keypad Row 1	Digital	I/O	Matrix scanning
D3	Keypad Row 2	Digital	I/O	Matrix scanning
D4	Keypad Row 3	Digital	I/O	Matrix scanning
D5	Keypad Row 4	Digital	I/O	Matrix scanning
D6	Keypad Column 1	Digital	I/O	Matrix scanning
D7	Keypad Column 2	Digital	I/O	Matrix scanning
D8	Keypad Column 3	Digital	I/O	Matrix scanning
D9	Keypad Column 4	Digital	I/O	Matrix scanning
D10	MOSFET Gate (Pump Control)	Digital Output (PWM-capable)	Output	Controls water pump
D11	Green LED	Digital Output	Output	Indicates pump active
D12	Red LED	Digital Output	Output	Indicates error condition

5. Libraries

5.1 Development Environment

The software is developed using:

- Visual Studio Code
- PlatformIO IDE Extension
- PlatformIO Core
- Arduino Framework
- C++17 compatible Arduino language subset

All source files are organized as a standard PlatformIO project.

5.2 PlatformIO Libraries

5.2.1 LiquidCrystal_I2C

Provides communication with the 16×2 LCD through the I2C interface. The LCD is responsible for:

- Home screen
- Main menu
- Settings pages
- Status screen
- Error messages
- User prompts
- Confirmation messages

Library: marcoschwartz/LiquidCrystal_I2C

5.2.2 Keypad Library

Provides scanning and debouncing of the 4×4 matrix keypad.

Library: Chris--A/Keypad

Features

- Matrix scanning
- Key debouncing
- Character detection
- Non-blocking operation

All keys on a 4*4 keypad (0-9,A,B,C,D,#,*) are supported.

5.2.3 EEPROM Library

The purpose of this is storing settings permanently. The following values survive power loss:

- Selected operating mode
- Moisture threshold
- Pump duration
- Watering interval
- Last saved configuration

5.2.4 Wire Library

The purpose of this is I2C communication. Used internally by the LCD library. No direct application logic is implemented using Wire.

5.2.5 Arduino Core

Standard Arduino framework.

Provides

- pinMode()
- digitalRead()
- digitalWrite()
- analogRead()
- millis()
- delayMicroseconds()
- EEPROM access
- timing functions

5.3 PlatformIO Configuration

The software is configured to target the Arduino Nano (ATmega328P) platform using the Arduino framework. Serial communication is configured with a baud rate of 9600 bps for debugging and monitoring purposes. The project depends on the LiquidCrystal_I2C library for controlling the LCD module and the Keypad library for interfacing with the 4×4 keypad.

6. PlatformIO Project Structure

The project follows the standard PlatformIO directory layout:

PlantWateringSystem/

├─ include/

├─ lib/

├─ src/

├─ test/

├─ docs/

├─ platformio.ini

└─ README.md

6.1 include/

The include/ directory contains all project-wide public header files shared between multiple modules. Files are:

- Config.h
- PinDefinitions.h
- Constants.h
- SystemTypes.h
- Version.h

These headers define:

- Compile-time configuration
- Hardware pin assignments
- Global constants
- Enumerations
- Shared data structures
- Version information

6.2 lib/

Contains reusable software modules. Each module is isolated inside its own folder.

- lib/
- LCDManager/
- KeypadManager/
- PumpController/
- SensorManager/
- MenuSystem/
- EEPROMManager/
- TimerManager/
- ModeManager/
- StatusManager/
- Hardware/

Every library contains exactly one public header (LibraryName.h) and one implementation file (LibraryName.cpp). Additional private helper files were added if required.

6.3 src/

Contains all application source code. Files:

- main.cpp
- Application.h
- Application.cpp

main.cpp only contains:

- setup()
- loop()

Responsibilities:

- Instantiate the Application object.
- Call the application initialization function from setup().
- Call the application execution function from loop().

No application logic, state machine implementation, hardware control, menu handling, timing logic, or business logic are implemented in main.cpp. All application behavior is implemented inside the Application class or classes owned by it.

6.4 test/

Contains all unit tests. Future versions of the project may use the PlatformIO Unity testing framework. Test code remains completely separate from production source code and does not introduce dependencies into the release firmware.

6.5 docs/

Contains project documentation. Typical contents include:

- Datasheets
- Circuit diagrams
- Wiring diagrams
- State machine diagrams
- User manual
- Project report

Files contained in this directory are reference documentation only and do not participate in compilation, linking, or firmware generation. Source code does not depend on any file located in this directory.

7. Folder Structure

7.1 Structure overview

The project organizes software into independent modules:

PlantWateringSystem/

include/

Config.h

Constants.h

PinDefinitions.h

Types.h

src/

main.cpp

Application.cpp

Application.h

lib/

LCDManager/

LCDManager.cpp

LCDManager.h

KeypadManager/

KeypadManager.cpp

KeypadManager.h

SensorManager/

SensorManager.cpp

SensorManager.h

PumpController/

PumpController.cpp

PumpController.h

EEPROMManager/

EEPROMManager.cpp

EEPROMManager.h

MenuManager/

MenuManager.cpp

MenuManager.h
TimerManager/
TimerManager.cpp
TimerManager.h
ModeManager/
ModeManager.cpp
ModeManager.h
StatusManager/
StatusManager.cpp
StatusManager.h
Hardware/
DigitalOutput.cpp
DigitalOutput.h
AnalogInput.cpp
AnalogInput.h

7.2 Folder Responsibilities

7.2.1 LCDManager

Responsible only for displaying information. It never makes application decisions.
Responsibilities:

- Screen drawing
- Cursor positioning
- Page switching
- Status messages
- User prompts

7.2.2 KeypadManager

Responsible only for keypad input. Responsibilities:

- Key scanning
- Debouncing
- Character detection
- Input buffering

7.2.3 SensorManager

Responsible for

- Reading soil moisture
- Filtering analog readings
- Returning moisture percentage
- Detecting sensor errors

7.2.4 PumpController

Responsible for

- Pump activation
- Pump deactivation
- Pump timing
- Safety timeout
- Preventing multiple simultaneous activations

7.2.5 EEPROMManager

Responsible for:

- Saving settings
- Loading settings
- EEPROM validation
- Default settings

7.2.6 TimerManager

Responsible for:

- All software timers
- Non-blocking timing
- Watering intervals
- Countdown calculations

Uses millis() only.

7.2.7 MenuManager

Responsible for:

- Menu navigation
- Cursor movement
- Page transitions
- User selections

7.2.8 ModeManager

Responsible for executing the currently selected operating mode. Modes include:

- Moisture Mode
- Timer Mode
- Idle Mode

The manager invokes the PumpController when watering conditions are satisfied.

7.2.9 StatusManager

Responsible for generating all system status information shown to the user, including:

- Current operating mode
- Configured moisture threshold
- Configured watering interval
- Configured pump runtime
- Current soil moisture percentage
- Pump state (ON/OFF)

- Tank capacity (stored as a configured value for display purposes)

This module prepares status data only. It does not directly control hardware or update the LCD.

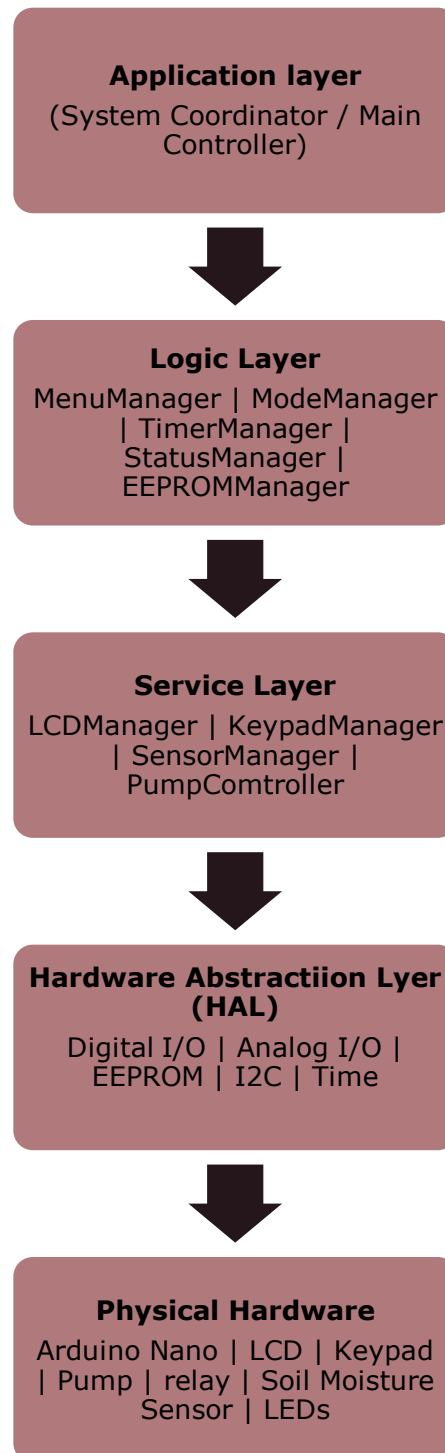
7.2.10 Hardware Layer

Contains low-level hardware wrappers. No application code accesses Arduino pins directly.

8. Software Architecture

8.1 Overview

The software follows a five-layer architecture to ensure modularity, maintainability, and scalability. Each layer has clearly defined responsibilities and communicates only with adjacent layers. This separation of concerns minimizes coupling and makes the system easier to test, debug, and extend.



8.2 Layer Responsibilities

8.2.1 Application Layer

Acts as the central coordinator. It initializes all modules, executes the main loop, schedules periodic tasks using `millis()`, and delegates work to the lower layers. It contains no direct hardware access.

8.2.2 Logic Layer

Implements the complete behavior of the system:

- Menu navigation and finite-state machine
- Operating mode selection
- Timer calculations
- Moisture threshold evaluation
- Settings management
- Status generation
- Decision making for automatic watering

This layer is entirely independent of Arduino-specific functions.

8.2.3 Service Layer

Provides reusable services that interact with hardware while hiding implementation details from the logic layer.

Services include:

- LCD rendering
- Keypad input processing
- Soil moisture acquisition and filtering
- Pump activation/deactivation with runtime control

These modules expose clean interfaces and perform no high-level decision making.

8.2.4 Hardware Abstraction Layer (HAL)

Encapsulates all direct interactions with Arduino hardware and core libraries. Responsibilities include:

- Digital pin reads/writes
- Analog-to-digital conversion
- I2C communication
- EEPROM read/write operations
- System timing via `millis()`

No upper layer directly calls Arduino functions such as `digitalWrite()`, `analogRead()`, or `pinMode()`.

8.2.5 Physical Hardware Layer

Represents the actual electronic components:

- Arduino Nano
- Soil moisture sensor
- MOSFET driver circuit
- Water pump

- 16×2 I2C LCD
- 4×4 matrix keypad
- Status LEDs
- External power supply

9. Finite-State Machine (FSM)

9.1 Purpose

The entire user interface is implemented as a deterministic finite-state machine (FSM). The system never uses deeply nested menus or blocking loops. Instead, every screen is represented as an independent state.

Each state:

- Draws its screen (only when entering the state)
- Reads keypad events
- Updates internal variables
- Decides the next state

The Arduino loop() function continuously executes:

Reads Inputs

↓

Updates Moisture Sensor

↓

Updates Pump Logic

↓

Updates FSM

↓

Updates LCD (only if needed)

↓

Repeats

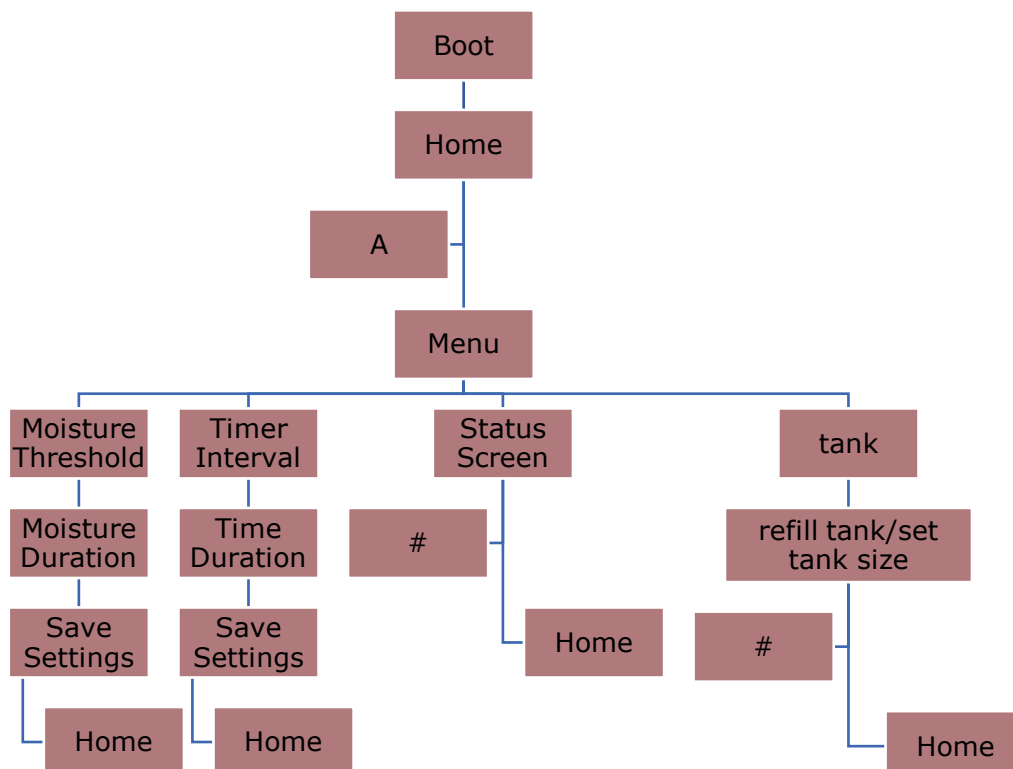
This guarantees responsive input while watering and sensor monitoring continue in the background.

9.2 Complete State List

State ID	Description
STATE_BOOT	System startup
STATE_HOME	Idle home screen
STATE_MAIN_MENU	Main menu
STATE_MOISTURE_THRESHOLD	Enter moisture threshold
STATE_MOISTURE_DURATION	Enter watering duration
STATE_TIMER_INTERVAL	Enter watering interval

State ID	Description
STATE_TIMER_DURATION	Enter watering duration
STATE_SAVE_SETTINGS	Save settings confirmation
STATE_STATUS	View system status
STATE_WATERING	Pump currently running
STATE_ERROR	Reserved for future errors
STATE_LOW_WATER	When the water remaining in the tank (which is calculated) is less than 250mL. In this case the water pump should be disabled.
STATE_TANK_MENU	Displays the tank management menu. Allows the user to choose between refilling the tank or setting the tank capacity.
STATE_REFILL_CONFIRM	Requests confirmation before resetting the remaining water amount to the configured tank capacity.
STATE-SET-TANK-CAPACITY	Allows the user to enter a new tank capacity value. The value is validated, stored in EEPROM, and becomes the maximum tank capacity.

9.3 FSM Diagram (Logical)



9.4 State Descriptions

9.4.1 STATE_BOOT

Executed only once.

Actions:

- Initializes LCD
- Initializes EEPROM
- Loads saved settings
- Initializes pump
- Initializes keypad
- Initializes moisture sensor
- Displays startup logo for 2 seconds

Next State: STATE_HOME

9.4.2 STATE_HOME

Purpose: Default idle state.

Continuously:

- Reads moisture sensor
- Checks timer
- Allows automatic watering (if the calculated water in the tank is above the limit)
- Waits for user input

Keypad:

A → STATE_MENU

9.4.3 STATE_MENU

Displays available options.

1. 1 Moisture
2. 2 Timer
3. 3 Status
4. 4 Tank

Keypad:

1 → Moisture setup

2 → Timer setup

3 → Status

4 → Tank

A → Home

9.4.4 STATE_MOISTURE_THRESHOLD

User enters: Moisture Threshold. Numbers 0 to 100 is valid. Entered digits are displayed live. Pressing # Confirms.

Next: STATE_MOISTURE_DURATION

9.4.5 STATE_MOISTURE_DURATION

- Displays the prompt "Pump Duration (sec)".
- Accepts a numeric input in the range 1–60 seconds.
- Pressing # confirms the entered value.
- Stores the confirmed value.
- Transitions to STATE_SAVE_SETTINGS.

9.4.6 STATE_TIMER_INTERVAL

- Displays the prompt "Water Every (min)".
- Accepts a numeric input in the range 1–999 minutes.
- Pressing # confirms the entered value.
- Stores the confirmed value.
- Transitions to STATE_TIMER_DURATION.

9.4.7 STATE_TIMER_DURATION

- Display the prompt "Pump Duration (sec)".
- Accept a numeric input in the range 1–60 seconds.
- Pressing # confirms the entered value.
- Store the confirmed value.
- Transition to STATE_SAVE_SETTINGS.

9.4.8 STATE_SAVE_SETTINGS

Operations:

- Validates values and writes to EEPROM (if changed).
- Records `displayStartTime = millis()`.
- Displays: "Settings Saved" (Row 1).
- FSM Check: Continuously checks `if (millis() - displayStartTime >= 4000ms)`.
- Goes to next State after 4 seconds: `STATE\_HOME`.

9.4.9 STATE_STATUS

Displays current configuration. There are 3 pages and the LCD changes through screens every 3 seconds. When # is pressed, goes back to Home Screen.

9.4.10 STATE_WATERING

Pump is ON. LCD shows watering message. The green LED is ON. When duration expires, both water pump and LED are turned off, and LCD goes back to STATE_HOME.

9.4.11 STATE_TANK_MENU_TRANSITION

This state is triggered when user presses the designated key for tank management. Then, LCD displays the tank management menu, and next state would be STATE_TANK_MENU.

9.4.12 STATE_TANK_MENU

The purpose of this state is to provide access to tank-related maintenance functions.
Transitions:

Condition	Next State
-----------	------------

User selects "Refill Tank"	STATE_REFILL_CONFIRM
User selects "Set Tank Capacity"	STATE_SET_TANK_CAPACITY
User presses Cancel	STATE_MAIN_MENU

9.4.13 STATE_REFILL_CONFIRM

The purpose of this state is to prevent accidental resetting of the remaining water value. Transitions:

Condition	Action	Next State
User confirms	RemainingWater = TankCapacity	STATE_MAIN_MENU
User cancels	None	STATE_TANK_MENU

9.4.14 STATE_SET_TANK_CAPACITY

The purpose of this state is to configure the maximum capacity of the connected water reservoir. Transitions:

Condition	Action	Next State
Valid capacity entered and confirmed	Store TankCapacity in EEPROM. Set RemainingWater = TankCapacity.	STATE_MAIN_MENU
Invalid value	Display error message.	STATE_SET_TANK_CAPACITY
User cancels	None	STATE_TANK_MENU

If the system triggers an automatic watering cycle while the user is in ANY menu state (not STATE_HOME), the LCD does not switch to STATE_WATERING. The pump and Green LED are activated in the background, allowing the user to finish their configuration uninterrupted.

10. EEPROM Memory Layout

All user-configurable settings are stored in EEPROM so they persist after power loss.

ADDRESS	TYPE	VARIABLE
ADRESS 0	uint8_t	Operating Mode (0 = Moisture, 1 = Timer)
ADRESS 1	uint8_t	Moisture Threshold (0–100%)
ADRESS 2	uint16_t	Timer Interval (uses 2 bytes: 2 & 3)
ADRESS 4	uint8_t	Pump Duration (1–60 seconds)
ADRESS 5	uint16_t	Tank Capacity (uses 2 bytes: 5 & 6)
ADRESS 7	uint16_t	Remaining Water (uses 2 bytes: 7 & 8)
ADRESS 9	uint16_t	Dry Moisture Calibration Value (0-1023 ADC) (This uses bytes 9 & 10)
ADRESS 11	uint16_t	Wet Moisture Calibration Value (0-1023 ADC) (This uses bytes 11 & 12)
ADRESS 13	uint16_t	EEPROM Signature (0x55AA)

10.1 EEPROM Signature

A 16-bit signature (magic number) is stored at the beginning of the EEPROM to indicate that valid configuration data has previously been written.

During system startup, the firmware reads the stored signature.

- If the stored signature matches the predefined value (0x55AA), all configuration parameters are loaded from EEPROM.
- If the signature does not match, the firmware assumes that the EEPROM does not contain valid configuration data. Factory default settings are loaded into RAM, written to EEPROM together with the signature, and then used for normal operation.

This mechanism ensures reliable detection of first-time startup or corrupted EEPROM contents.

10.2 Factory Defaults

Mode: Moisture

Threshold: 40%

Interval: 180 minutes

Pump duration: 5 seconds

11. Moisture Sensor Processing

11.1 Reading Method

The sensor is connected to one Arduino analog input. Sampling occurs continuously in the main loop at a fixed interval (e.g., every 500 ms) without blocking other tasks.

11.2 Filtering

To reduce noise, a moving average is used over the last 10 samples:

$\text{filteredValue} = (\text{sample1} + \text{sample2} + \dots + \text{sample10}) / 10$

Only the filtered value will be used by the control logic.

11.3 Calibration

The raw ADC reading (0–1023) is mapped to a moisture percentage (0–100%) using calibrated dry and wet reference values:

Dry calibration → 0%

Wet calibration → 100%

These calibration constants are defined in configuration headers so they can be adjusted without modifying the processing logic. Calibration constants are hardcoded as constexpr in Config.h

11.4 Moisture Mode Decision

When operating in Moisture Mode:

If ($\text{filteredMoisture} < \text{threshold}$)

then a watering cycle is requested.

To prevent rapid on/off switching due to sensor noise, implement hysteresis. Watering is not requested again until the measured moisture has risen above:

$\text{threshold} + \text{hysteresis}$

A hysteresis value is configurable.

11.5 Sensor Fault Detection

The software detects implausible sensor readings, such as values outside the expected calibrated range or readings that remain unchanged for an unusually long period, and enters an error state or ignores the reading. This behavior is reserved for future implementation but the processing module is structured to support it.

12. Pump Control Logic

12.1 General description

Pump control is completely non-blocking. The pump is not controlled using `delay()`. Timing is based on `millis()` so that keypad input, LCD updates, and sensor processing continue while watering is in progress.

12.2 Pump Activation

A watering cycle is initiated by:

1. Moisture Mode: filtered moisture falls below the configured threshold (subject to hysteresis and cooldown).
2. Timer Mode: the configured watering interval has elapsed since the previous watering cycle.

Only one watering cycle may be active at any given time.

12.3 Watering Cycle

When a watering request is accepted:

1. The start time is recorded using `millis()`.
2. The relay turns on.
3. System enters `STATE_WATERING`.
4. Normal executions in the main loop are continued.

The pump remains ON until:

`millis() - pumpStartTime >= pumpDuration × 1000`

At that point:

- Relay turns off.
- Completion time is recorded (used by Timer Mode).
- System return to `STATE_HOME`.

12.4 Tank Volume Estimation

After the pump turns OFF, the remaining water is updated using

$$waterUsed = pumpLowRate \times pumpRunTime$$

Where `pumpRunTime` is the time duration the water pump is on and is configured by the user. `pumpFlowRate` is in mL/sec and is defined as a software constant. This is defined 30ml/sec but is configurable. After that we have:

$$remainingWater -= waterUsed$$

The remaining water value is stored in EEPROM after every watering cycle. If the calculated water remaining in the tank is less than 250mL, the water pump stops working, the LCD writes "Row1: Low Water / Row2: Remaining" and the red LED turns on. When key "C" is pressed, then, in that case, the `remainingWater` is changed to `tankCapacity`, and then the system goes back to functioning normally with the pump watering the plant with the configurations entered by the user again. The key "C" only works for changing the value of the water left in the tank if the system is in `STATE_HOME` or `STATE_LOW_WATER`.

12.5 Cooldown Protection

To avoid repeated watering caused by unstable sensor readings, a configurable cooldown period is implemented after every watering cycle. During this period, additional watering requests are ignored. A default cooldown is 30 seconds.

12.6 Mutual Exclusion

If a watering cycle is already active:

- Any new moisture-triggered request is ignored.
- Any timer-triggered request is ignored.

12.7 Safety on Startup

Immediately after power-up, the system,

- initializes the pump control pin as an OUTPUT.
- drives the pin LOW before enabling any other subsystem.
- loads configuration from EEPROM.
- Does not start watering automatically until at least one complete control cycle has executed and a valid sensor reading has been obtained.

This ensures the pump can never energize unintentionally during boot.

13. Algorithms

13.1 Timer Mode Algorithm

The system automatically waters the plant at a fixed user-defined interval.

Configuration Parameters

- Watering Interval (minutes)
- Pump Duration (seconds)

Algorithm

1. Watering interval and pump duration are loaded from EEPROM.
2. The completion time of the previous watering cycle is recorded.
3. The system continuously compares the elapsed time with the configured interval.
4. When the interval expires, the system:
 - Verifies that the pump is not already running.
 - Starts a watering cycle.
5. After watering finishes, the system:
 - Updates the last watering timestamp.
 - Restarts interval timing.
6. Continue indefinitely until the operating mode changes.

The timer is implemented using `millis()` and it never uses `delay()`.

13.2 Moisture Mode Algorithm

The system waters the plant only when the measured soil moisture falls below the configured threshold.

Configuration Parameters

- Moisture Threshold (%)
- Pump Duration (seconds)

Algorithm

1. Read the moisture sensor periodically.
2. Filter the readings using a moving average.
3. Convert the filtered value to a moisture percentage.
4. Compare the measured value with the configured threshold.
5. If:

Moisture < Threshold

The system starts a watering cycle.

6. After watering, the system:
 - Applies a cooldown period.
 - Requires the moisture level to rise above the threshold plus hysteresis before allowing another watering cycle.

This prevents rapid repeated watering caused by sensor noise.

13.3 Tank Volume Algorithm

When watering begins

↓

Pump runs

↓

Duration expires

↓

$\text{WaterUsed} = \text{PumpFlowRate} \times \text{Duration}$

↓

$\text{RemainingWater} -= \text{WaterUsed}$

↓

Clamp to zero

↓

The system saves RemainingWater to EEPROM

↓

If $\text{RemainingWater} < 250 \text{ mL}$

Red LED is on

↓

Otherwise, red LED is off

14. Startup Sequence

After power is applied, the software performs the following initialization sequence:

1. Configures all GPIO pins.
2. Ensures the pump output is OFF.
3. Initializes the LCD.
4. Displays the startup screen for 2 seconds.
5. Initializes the keypad.
6. Initializes the moisture sensor.
7. Loads settings from EEPROM.
8. If EEPROM is uninitialized, factory defaults are loaded.
9. Initializes timers and internal variables.
10. Enters the Home screen.

15. Shutdown Behavior

Since the Arduino Nano has no operating-system-controlled shutdown, the software only guarantees a safe state when power is removed.

Before power loss (normal operation):

- Pump is only ON during an active watering cycle.
- All user settings are already stored in EEPROM immediately after modification.

If power is removed:

- The pump immediately stops.
- Volatile variables are lost.
- Configuration remains stored in EEPROM.

On the next startup, the previous configuration is restored automatically.

16. Error Handling

The software includes basic validation and fault handling to ensure stable operation.

16.1 Invalid User Input

If an entered value is outside the allowed range:

- System displays "Row1: Invalid Value/ Row2: Try Again" for 3 seconds.
- Returns to the same input screen.

16.2 EEPROM Initialization Failure

If no valid EEPROM signature is found, factory default settings are loaded.

16.3 Sensor Fault (Future Support)

The software architecture allows detection of:

- Sensor disconnected
- Sensor short circuit
- Invalid ADC values
- Constant readings for an extended period

For this project version, these faults simply prevent automatic watering until valid readings return.

16.4 Low Water Condition

This happens when the water left in the tank (which is calculated) is less than 250mL. In this case the water pump is disabled. The Red LED turns on, and the LCD writes "Row1: Low Water Row2: Remaining". This should be it until key "C" is pressed, which means refilling water tank. If the system is on STATE_HOME or STATE_LOW_WATER, in that case pressing "C" key will change the value remainingWater to tankCapacity. In that case, the system goes back to the same state as before, when the pump was working normally and was watering the plant. Notice there are 2 ways the user can tell the system that the water tank is filled, one is through menu path, and the other is by pressing key "C" any time.

16.5 Pump Protection

The software ensures:

- Only one watering cycle can be active at a time.
- New watering requests are ignored while the pump is running.
- The pump output is forced LOW during initialization and whenever the system is idle.

These protections prevent unintended pump activation and ensure reliable system operation. If any of these errors occur, the plant watering system will stop working and the Red LED turns ON and the green LED turns OFF.

17. Input Validation

All user inputs are validated before being accepted or stored in EEPROM.

Parameter	Valid Range
Moisture Threshold	0–100 %
Watering Interval	1–999 minutes
Pump Duration	1–60 seconds
Tank Capacity	1-9999 mL

Validation rules:

- Only numeric characters (0–9) are accepted during value entry.
- A maximum of three digits may be entered, except for tank capacity which upto four digits are valid.
- # confirms the input.
- deletes the last entered digit.
- Values outside the allowed range shall be rejected.
- Invalid values shall display " Row1: Invalid Value/ Row2: Try Again" for 3 seconds, after which the user returns to the same input screen.
- EEPROM shall only be updated after successful validation.

18. Memory Constraints

The implementation is optimized for the Arduino Nano's limited resources.

SRAM

- Avoids unnecessary global variables.
- Avoids duplicate data buffers.
- Stores constant text in Flash memory using F() where appropriate.
- Does not use dynamic memory allocation (new, delete, malloc, free).

Flash Memory

- Organizes code into reusable modules to minimize duplication.
- Removes unused variables, libraries, and functions.
- Uses constexpr constants where appropriate.

EEPROM

Only persistent configuration values are stored:

- Operating mode
- Moisture threshold
- Watering interval
- Pump duration
- EEPROM signature
- Current estimated tank volume
- Configured full tank capacity

No temporary runtime variables are written to EEPROM.

EEPROM writes only occur when settings are changed to minimize write cycles.